



SECURITY POLICY

Security and Privacy

Reporting, trust boundaries, network exposure, shutdown, and release integrity.

2026 EDITION / IAIN BENNETT / MIT LICENSE

Use VirtualGlove on a trusted home or workshop network. This policy explains how to report a vulnerability and which protections the project expects pairing, networking, and shutdown code to maintain.

Supported code

Security fixes will be available on the current [main](#) branch and included in the next tagged release. Older snapshots may lack pairing, network, dependency, or shutdown protections and should be upgraded before troubleshooting them.

Reporting a vulnerability

Please do not publish credentials, pairing codes, tokens, private device configuration, or working exploit instructions in a public issue.

1. Open the repository's **Security** tab and look for private vulnerability reporting.
2. Submit a private report with the information below. Remove secrets from every attachment.
3. If private reporting is unavailable, open a minimal public issue asking for a private contact channel; include no exploit details or secrets.

Include these details in the private report:

- the affected commit or release;
- the VirtualGlove Controller, selected console platform, browser, and network environment involved;
- concise reproduction steps and the observed result;
- the security boundary that was crossed;
- logs or screenshots after removing tokens, passwords, pairing codes, local addresses, and unrelated personal information.

This project does not currently offer a bug bounty or a guaranteed response time.

You can report controller-mapping, camera-compatibility, and game-profile problems in public issues after removing sensitive information from the logs.

Security model

The **VirtualGlove Controller** is the Arduino UNO Q device that owns the camera, recognition pipeline, local website, and controller sender.

VirtualGlove is designed for a trusted home or workshop network. The VirtualGlove Controller performs hand tracking and sends virtual-controller state to the paired console. The console sends per-game profile changes back to the VirtualGlove Controller. Neither device should be treated as an Internet-facing service.

The main protected assets are:

- the shared controller token;
- the VirtualGlove Controller and console operating systems;
- the console receiver and its virtual-input interface;

- the physical pairing display and single-use PIN;
- the fixed-purpose VirtualGlove Controller shutdown and USB-camera recovery helpers;
- the integrity of the App Lab installation ZIP, MediaPipe wheel, bundled or downloaded model, and Arduino dependencies.

The project does not attempt to protect a device after an attacker obtains root access, physical storage access, or control of the trusted local network and both paired hosts.

Pairing boundaries

Controller Router owns the UNO Q-console relationship. VirtualGlove and R.O.B. Vision receive independent, revocable credentials; neither can use the other app's credential. Router's management credential remains in its private connection registry. Product credentials remain in each app's private settings and its console token file, never browser responses, process arguments, diagnostics, or logs.

Fresh pairing uses Router's HTTPS Setup on port **8444**. The user checks the UNO Q certificate fingerprint, supplies the console's **CR1 connection code**, and enters the six-digit PIN delivered to the physical Matrix. The console code pins its TLS certificate before any code or credential is transmitted. Console codes expire after five minutes and are single-use; Matrix confirmation expires after two minutes. Five incorrect attempts close the current confirmation window. No SSH password is collected.

The console's persistent TLS management service uses port **55359**; opening a pairing window grants only temporary authorization to provision a connection. Requests have bounded sizes, timeouts, and concurrency. Browser writes require matching HTTPS Origin, JSON, and the pairing action header. These protections preserve the trusted-LAN administration model; they do not create browser user accounts.

Router creates stable local certificates with private keys restricted to mode **0600**. The installer prints the public fingerprint. Users may trust the downloaded public certificate on their own browser device after checking it. Certificate mismatches require investigation or an explicit new pairing; they are not accepted silently.

Migration binds a fresh HMAC proof to the observed console identity and certificate. It never consolidates records solely by hostname. Conflicting records remain intact for an explicit physical re-pairing. Private recovery journals retain prior credentials and configuration; failed provisioning restores the previous connection, or reports that service recovery is still required.

Pairing and access changes are blocked during a live game. Installing another supported app provisions its own access over the existing authenticated relationship. Individual revocation leaves the other app connected. Complete removal revokes Router and both app credentials. Pairing does not rewrite saved RetroArch configuration.

The supervised vision worker reads credentials from private device settings. Signed profile renewals are checked against the console-specific credential and replies use that same credential. The console stores its VirtualGlove credential in a restricted token file; `launcher.json` records its path and destination only. Older product-specific pairing code remains a compatibility path for installations without the shared Router authority.

Console input boundaries

On Recalbox and Batocera-and on RetroPie only when explicitly enabled-the root-owned Controller Router reads only explicitly saved EmulationStation mappings and the receiver's bounded local datagrams. It emits up to four fixed-capability devices named **VirtualGlove Merged Player 1-4**. Only Player 1 carries a physical hotkey; VirtualGlove Select has no path to that control. Malformed mappings and local states are rejected, tracking timeout clears only the VirtualGlove source, and a physical disconnect releases only that source. Outputs are neutral and physical devices are not grabbed outside Libretro gameplay on Router-enabled systems, so Router cannot duplicate EmulationStation navigation. Its Unix socket and versioned record remain inside the platform's private VirtualGlove directories. RetroPie and Batocera use temporary session routing settings; runtime does not rewrite saved RetroArch configuration. Core and game overrides can load afterward and replace those settings. Nestopia (VirtualGlove) uses a separate native-input path for gestures; it never admits VirtualGlove as a duplicate RetroPad source.

On every supported-core transition, Router clears the stored VirtualGlove source and requires a new neutral D-pad/button observation before admitting gesture input. Physical sources are not gated. Camera position axes are not published through the ordinary NES joystick route, preventing pre-launch or off-centre camera state from becoming a game-start input.

LaunchBox installs no Windows virtual-pad, keyboard-filter, or device-hiding driver. Ordinary games receive VirtualGlove through RetroArch's built-in Network RetroPad on a random high UDP port. The sender targets [127.0.0.1](#), the interface is enabled only by the managed FCEUmm append configuration, and a managed Windows Firewall rule blocks LocalSubnet access to that RetroArch port. Installation fails closed if the rule or loopback transport does not validate. The signed VirtualGlove receiver remains the only LAN-facing input endpoint. Physical XInput and real keyboard bindings stay independent; keyboard-command conflicts are reported only as degradation of that manual fallback.

Network exposure

Port	Protocol	Direction	Boundary
55355	UDP	VirtualGlove Controller to console	Authenticated virtual-controller packets
55356	UDP	Console to VirtualGlove Controller	HMAC-authenticated profile commands and acknowledgements
55357	TCP/TLS	Pairing client to temporary server	Short-lived code-pairing exchange only
8100	HTTP	Browser to VirtualGlove Controller	Local dashboard, Play, public Help guides, diagnostics, and ordinary controls; no pairing credentials accepted
8443	HTTPS	Browser to VirtualGlove Controller	Protected setup and pairing operations
80	HTTP	Browser to shared Controller Router	App selection and shared routing Setup on the trusted LAN

Keep these ports on a trusted LAN. Do not configure router port forwarding, public reverse proxies, cloud tunnels, or Internet firewall exceptions for them. Guest Wi-Fi and untrusted shared networks are inappropriate unless the devices are isolated by firewall rules or a dedicated VLAN.

The profile-control brick publishes UDP [55356](#) and forwards packets to the worker on the private container network. It has no token or private data mount, runs without elevated privileges, and bounds packet sizes, pending exchanges, and reply lifetime. Authentication remains in the worker; the relay never creates an acknowledgement. A signed acknowledgement confirms queue admission, not completed camera startup or enabled gameplay delivery.

Controller and profile UDP traffic is authenticated but not encrypted. Anyone with access to the local network can observe packet timing and size even when they cannot create accepted input without the token. The dashboard can expose camera imagery and operational status to clients that can reach it, so network access to port 8100 is itself sensitive.

The Help library serves a fixed list of public Markdown guides and images from the installed application. It must not expose `data/`, the machine-specific cheat sheet, arbitrary filesystem paths, or pairing credentials. Markdown HTML is not executed, unsafe link schemes are rejected, and image requests are confined below `docs/images`.

The dynamic **This console** page accepts only a validated hostname or IP from the browser `Host` header and combines it with `public_config()`. It may show local network addresses, ports, profile selection, camera selection, and whether pairing is configured, but it must never return the token value, passwords, private files, or arbitrary Host-header content.

Shutdown and camera-recovery permissions

The web process does not receive general `sudo` permission. A root-owned systemd path unit watches one fixed file in the application data directory. A matching oneshot service deletes that file and requests a non-blocking Linux halt.

The dashboard route requires an explicit confirmation header and only creates the fixed request when the host installer has placed the private `.shutdown-enabled` marker. These checks reduce accidents and prevent command substitution; they do not make the dashboard safe for public network exposure. Anyone able to use the reachable dashboard may still cause a denial of service by shutting down the VirtualGlove Controller.

A root-owned tmpfiles rule recreates only that fixed readiness marker during boot. It grants no command execution and does not change the container's privileges.

Camera recovery follows the same fixed-request pattern with separate path and service units. Installation may occur without a camera. When exactly one UVC camera is healthy, the root-owned helper writes its identity and its actual parent hub's identity and physical USB path to the root-owned `/etc/virtualglove-camera-recovery.json` allowlist. A later healthy sighting safely updates that association if the camera has moved. During an outage the helper validates both the stored path and hub identity and resets only that hub; it never accepts a device path from the web application or guesses among hubs.

The helper consumes the request before acting, permits one request per camera outage, and enforces a root-owned cooldown. Before first enrollment it refuses to reset anything. A client able to activate vision could still cause one brief USB interruption during a real camera outage, so the web interface remains suitable only for a trusted LAN.

Keep both helpers' path units, service units, scripts, and tmpfiles rules owned by root. Unit and rule permissions are 0644; the camera helper is 0755. Do not replace the fixed `ExecStart` commands with user input, a shell string, or an arbitrary command runner. Remove or disable the corresponding helper set if remote shutdown or camera recovery is not wanted.

Dependency and release integrity

- The App Lab installation ZIP is generated and verified; it is not maintained as a changing source-controlled binary.
- The custom MediaPipe wheel's provenance and checksum are recorded in [THIRD_PARTY_NOTICES.md](#) at the repository root.
- Google's Hand Landmarker model is installed from the bundled copy, with its pinned download as a fallback only when that copy is absent. Both paths must match the expected SHA-256 digest before atomic installation; the package verifier also checks the bundled model and license text.
- The optional Nestopia core is built from one pinned upstream commit and checksum-recorded patches. Recalbox and Batocera manifests bind each binary to its target, release, corresponding source archive, source revisions, size, digest, and executable identity; LaunchBox binds its DLL and source to a Windows manifest. Console installers load-test the core and confirm the libretro identity before activation. Batocera and LaunchBox fail safely to FCEUmm when the native artifact is absent, changed, or unloadable. Stock Nestopia and FCEUmm are not replaced.
- Arduino library versions are pinned in [sketch/sketch.yaml](#).
- GitHub Actions rebuilds and inspects documentation and the App Lab installation ZIP on every pull request and push to [main](#) or [dev](#).
- The public website is generated as static files from tracked source and bounded release facts. It contains no analytics, secrets, pairing material, or live Controller endpoint; its build rejects stale release and retired retired guided-readiness claims before creating the manual-upload ZIP.

Changing a download URL, checksum, dependency source, pairing primitive, network binding, file permission, or privileged service requires focused review and corresponding tests and documentation.

Paired games, controller routing, and gesture tuning

The separate console administration service listens on TCP [55358](#). Only the paired Controller proxy uses it; browsers call the UNO website. Requests and replies use a distinct HMAC-authenticated protocol. [/registry](#) uses [virtualglove-games/1](#); [/inputs](#) uses the separately domain-labelled [virtualglove-inputs/1](#) protocol for bounded inventory, read, save, check, and rollback operations. It returns friendly controller metadata and assignments, never device paths or pairing material. Server challenges expire after fifteen seconds and are consumed once. The shared token never goes to the browser. This protects message integrity; the LAN transport does not encrypt ROM filenames.

The service accepts only registry reads, validated writes, and restoration. File locations come from the administrator's launcher configuration, never from browser input. Writes use revision checks, atomic replacement, and a previous valid backup. The service has bounded document sizes, pending challenges, and socket timeouts; it runs separately from controller input delivery. Its systemd unit confines writes to the configured registry directory and removes device access and capabilities.

The Games, Controller Router, and Tune browser actions require JSON, an explicit action header, and matching Origin when supplied; cross-site browser requests are rejected. They retain the existing trusted-LAN administration model, not per-user accounts. Normal personalisation contains numerical thresholds only. Measurements are held briefly in memory, previews expire with the owning session, and camera images are not saved. Tuning suppresses controller delivery even if a game launches or another Dashboard requests input. Saved settings are validated and atomically replaced.

Dashboard rapid-fire changes are written through that revision-checked Games service before being forwarded to the local worker. A live A/B override is accepted only when the named game still matches an active authenticated launch lease. It is bound to that opaque session and cleared on game exit, lease expiry, manual profile selection, or the next launch; it never

changes the authenticated profile, destination, or pairing material.

The optional Advanced diagnostic is the only Academy path that records video. It is explicitly started and user-paced, remains on the VirtualGlove Controller, and is deleted immediately after aggregate analysis or cancellation. An abandoned AVI expires after 30 minutes. Its downloadable JSON contains aggregate continuity, latency, confidence, lighting, and recognised-state names only: no frames, landmarks, tokens, addresses, or saved personal thresholds.

Documentation screenshots

Documentation screenshots use isolated sample data and an omitted-camera placeholder. If capturing a live page instead, blur the complete camera image before capture. Keep controls legible, but never publish unblurred camera frames or screenshots that contain passwords, private tokens, or pairing codes. The reference images show the interface; they are documentation examples only.

Personal hand setup and tuning data

Optional hand setup measures all five fingers; gesture tuning measures selected components. Both use three short sets of numerical samples in memory. The version-6 [data/gesture-tuning.json](#) file stores player names, one joystick centre-box size, gesture activation/release pairs shared across game profiles for each player, Academy progress, and a required-centre flag and separate saved calibration, plus a bounded pending reference during a calibration restore. VirtualGlove v0.4.2 is the oldest supported in-place upgrade to 0.6.0 and already uses this store format. Portable hand-setup exports use the [virtualglove-hand-setup](#) format at version 4; older formats are rejected without changing their source or the active player. Exports contain a name, centre-box size, personal and complete gesture threshold pairs, software identity, and a neutral reference. They exclude camera images, landmarks, Wi-Fi credentials, pairing tokens, and lesson progress. Restoring calibration requires an explicit same-position confirmation and strict finite field validation. A persisted restore resumes after interruption with output gated; Start controller is still required. The active physical reference remains in [data/calibration.json](#); each player retains a separate saved reference and reuse requires confirmation. The installer never packages a maintainer's neutral reference: camera position, player distance, and wrist pose make it installation-specific. Preserve both files during updates. Expiry or discard removes temporary preview state, not saved settings. Numerical validity and sample separation do not prove a pose was performed correctly; preview feedback and physical testing are still required before release.

Wi-Fi status sampler

The network sampler runs as [arduino](#) and reads physical-link carrier state plus the subnet broadcast address for each connected Wi-Fi/Ethernet interface. It publishes a small expiring record in [data/wifi-status.json](#); it does not collect SSIDs, host unicast addresses, passwords, or scans, and cannot change network settings. The application retains no controller states while hostname resolution runs in the background. Controller version-2 messages use the signed sessions described below.

Controller packet integrity and replay boundaries

Version-2 controller messages use HMAC-SHA256 with a controller-specific domain prefix, separate from profile and registry messages. The secret is never included in these datagrams. All message fields are authenticated; size limits and duplicate-key rejection bound parsing. A receiver-issued random challenge is required for input, is bound to the sender session and UDP peer, and is replaced on session activation or receiver restart. Strictly increasing sequences reject duplicates and reordering within the current session. Old handshakes cannot resurrect recorded state because a fresh challenge needs a fresh authenticated response. Pending challenges are bounded and expire; input states are never queued by the handshake.

This protects message integrity and retired-session replay, not confidentiality or availability. A host holding the shared token can create valid input, and a network attacker can still drop traffic. Input fields remain readable on the LAN. The existing timeout neutralizes both gamepad and native state even under rejected traffic or repeated handshakes.

Address recovery does not trust a hostname, DHCP lease, or responding IP as the console's identity. When directed delivery or name resolution stops producing authenticated maintenance replies, the Controller may send the signed [hello](#) message to the broadcast address of each connected physical LAN. It accepts a new address only from a correctly signed, request-matched challenge made with the existing pairing key. State messages are never broadcast. Discovery is limited to the local broadcast domain; routers, guest isolation, VLANs, and firewalls may prevent it, which is a safe availability failure rather than an authentication bypass. A copied pairing key can impersonate the console and must be rotated if exposed.

The reverse profile path follows the same identity rule. When the console cannot reach the configured Controller address, it broadcasts a signed discovery request that contains no ROM or requested profile. It accepts only a signed, request-matched acknowledgement, sends the profile command to that authenticated source by unicast, and retains the address only in a bounded 30-second process cache. Neither direction rewrites saved addresses automatically.

The public system-report endpoint is field-allowlisted. It excludes network addresses as well as frames, secrets, ROM names, player identities, calibration, and gesture measurements; adding a new internal status field does not make that field appear in a report automatically.

The [architecture timing table](#) distinguishes gameplay packets, liveness handshakes, profile leases, Setup health checks, and host-link sampling. In particular, handshake or rejected traffic cannot extend the receiver's 250-millisecond valid-state deadline, and background status checks do not run in the camera inference path.

The receiver accepts signed protocol version 2 only. Unsigned and malformed packets are rejected before native-state or virtual-gamepad publication. Update both computers together; the sender never downgrades.